Project One: Selected Questions

Team Fitness · Movement Quest · CSC 4360

Mario Leonel Gutierrez Espinal (solo team). Undergraduate pathway, 12 questions.

I1 · Widget tree & ownership (Required)

Show one reusable StatelessWidget and one StatefulWidget in your app. Explain their responsibilities, how build() renders current values, and which interaction requires mutable state.

Evidence: widget tree, actual classes, and a before/after UI action.

I2 · Buttons, controls & state (Required)

Trace a button and slider interaction (or justify your numeric-control alternative) through input validation, setState()/state notification, and rendering. Which values are transient and which must be saved?

Evidence: callbacks, form validation, visible values, and saved settings/records.

A1 · Responsibilities

Trace one feature through widget, state/model, and data helper. Where are domain rules and database calls owned, and what would break if the widget performed every responsibility?

Evidence: architecture diagram and file-level walkthrough.

A2 · Lifecycle

Show how a controller, timer, listener, or async operation is owned and cleaned up. What happens when a user leaves the screen before an operation finishes?

Evidence: initState/dispose or equivalent ownership code and a navigation-away test.

D1 · Both entities' CRUD (Required)

Show create, read, update, and delete for your primary record and linked history record. Trace SQL/helper calls, validation, delete confirmation, and the final refreshed UI. Explain what happens to child records when a parent is deleted.

Evidence: schema/keys, parameterized operations, and both complete CRUD demonstrations.

D2 · Persistence & no-cloud proof (Required)

Explain SQLite versus SharedPreferences in your app. With networking disabled, save records and a setting, close/relaunch the app, and show restored values. Why are GitHub, ClickUp, and YouTube not app backends?

Evidence: dependencies, local file/schema ownership, airplane-mode restart, and restored setting.

D4 · Schema evolution

Design or demonstrate a schema change without silently losing saved history. How do versioning, migration, and test fixtures preserve record relationships?

Evidence: migration code or clearly labeled design, old/new schema, and verification plan/results.

D5 · Filter & aggregate

Show your topic filter and history count/total/average/completion rate. How do edited/deleted records affect derived values and empty-data behavior?

Evidence: query/calculation and a manually checkable dataset.

T1 · Acceptance coverage

Map tests to both entities' CRUD, the five-screen workflow, settings persistence, and offline operation. Show normal, empty, and invalid cases on two devices/emulators.

Evidence: test matrix, device details, and pass/fail results.

T2 · Failure & recovery

Show a real bug or deliberately tested save/lifecycle failure. What causes it, what feedback appears, and how does the user recover without duplicate or lost records?

Evidence: reproduction steps, fix or tested guard, and regression result.

R4 · Required smart helper & critical thinking (Required)

Demonstrate your local helper's input, suggestion, explanation, override, and no-data/invalid feedback. Why did you choose these inputs and this method? Compare with your baseline, show a counterexample, and justify a revision or retained limitation. Undergraduates show five case types; graduates additionally show two-method results on at least five held-out valid cases. Label rules versus ML accurately.

Evidence: AI_FEATURE.md, expected/actual case table, metric/sample size, local computation and offline/restart, user control, and individual contribution. AI_Usage_Log.md is separate and needed only if an AI development assistant was used.

R5 · Next iteration

Which planning choice created a limitation, and what would you change first? Propose an improvement that preserves local-only storage and equal topic scope.

Evidence: decision log, prioritized task, owner, and verification plan.